MERL
Accelerating Engineering Innovation

MERL Training Sessions

GENIFY
LECTURE 08
31-July-2025

TOPICS COVERED:

RIS

- ARTIFICIAL NEURAL NETWORKS

Presenter
Shahzaib Kashif

Co-Presenter
Shayan Hassan Baig

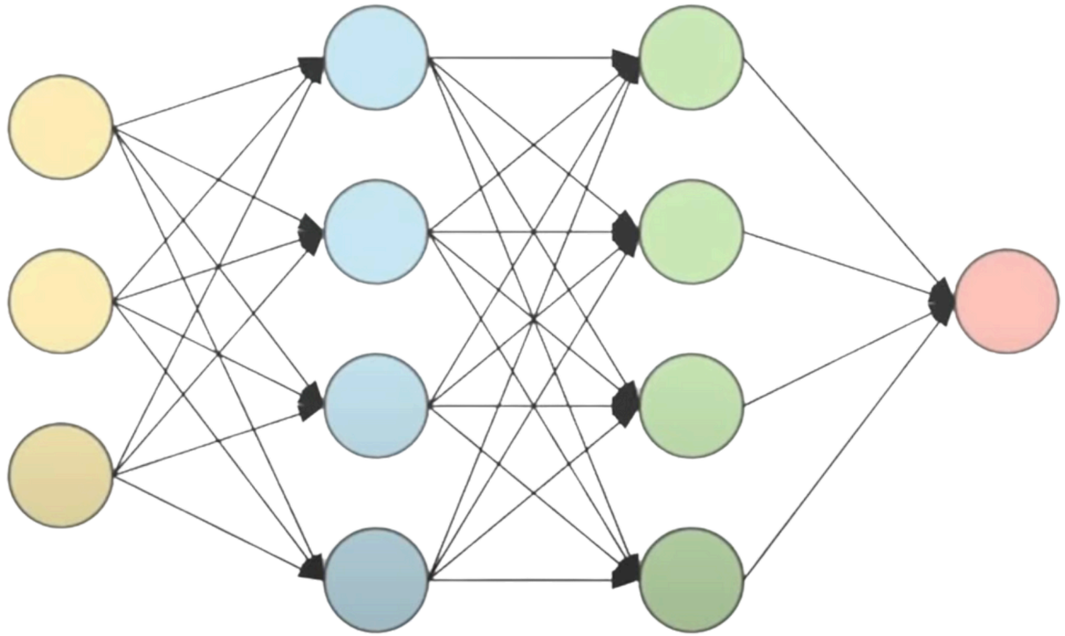
RECAP

QUIZ TIME

ARTIFICIAL NEURAL NETWORKS

Neural Network Basics Recap

- Neural networks are machine learning models that mimic the complex functions of the human brain.
- These models consist of interconnected nodes or neurons that process data, learn patterns and enable tasks such as pattern recognition and decision-making.
- **Questions:** Layers, Neurons, Activations, Weights, Weight adjustment (cf), Forward and Backwards Propogation.



How a Neural Network is Trained

Step 1 : Initialize weights randomly

Step 2 : Forward Propagation

$$Z1 = W1 * A0 + B1$$
$$A1 = f(Z1)$$

$$Z2 = W2 * A1 + B2$$
$$A2 = f(Z2)$$

Step 3 : find value of Cost

Step 4 : Backward Propagation

$$W2 = W2 - \text{alpha} * \frac{\partial \text{cost}}{\partial W2}$$

$$B2 = B2 - \text{alpha} * \frac{\partial \text{cost}}{\partial B2}$$

$$W1 = W1 - \text{alpha} * \frac{\partial \text{cost}}{\partial W1}$$

$$B1 = B1 - \text{alpha} * \frac{\partial \text{cost}}{\partial B1}$$

Step 5 : Repeat Step 2, 3 and 4 for many times

Note: W1 is matrix containing all weights associated to first layer and * represents matrix multiplication

Activation Functions

- **Question:** Why do we need Activation Functions?

$$Z1 = W1 * A0 + B1$$

~~$A1 = f(Z1)$~~

$$Z2 = W2 * A1 + B2$$

~~$A2 = f(Z2)$~~

$$Z3 = W3 * A2 + B3$$

~~$A3 = f(Z3)$~~

$$A2 = W2 * (W1 * A0 + B1) + B2$$

$$A2 = W2 * W1 * A0 + W2 * B1 + B2$$

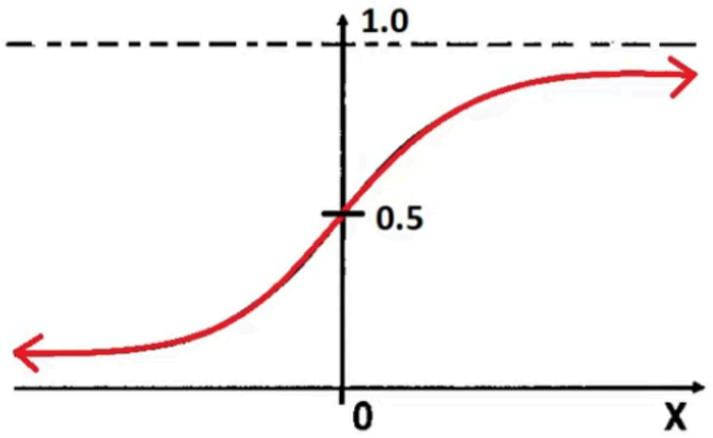
$$A2 = W' * A0 + B'$$

$$A3 = W'' * A0 + B''$$

- Hence we need functions to make these equations non linear.

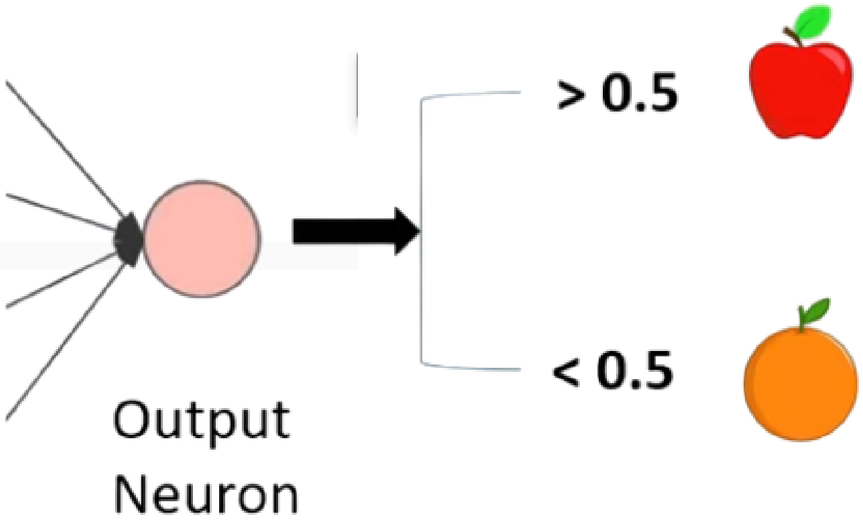
Types of Activation Functions

- **Sigmoid Function:**



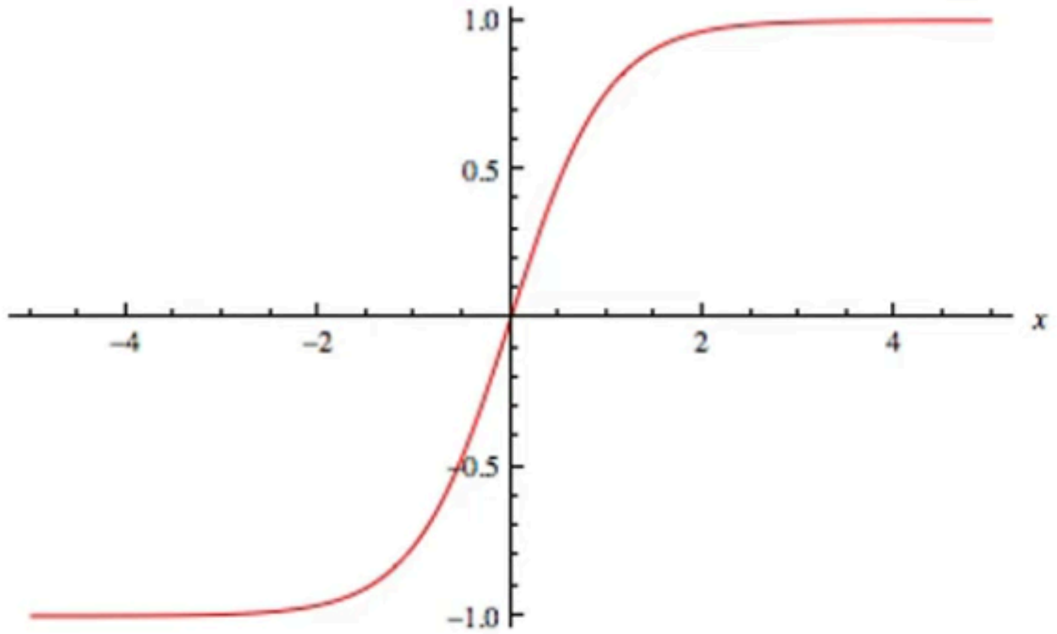
Sigmoid

$$f(x) = \frac{1}{1+e^{-x}}$$



Types of Activation Functions

- **TanH Function:**



$$\tanh = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

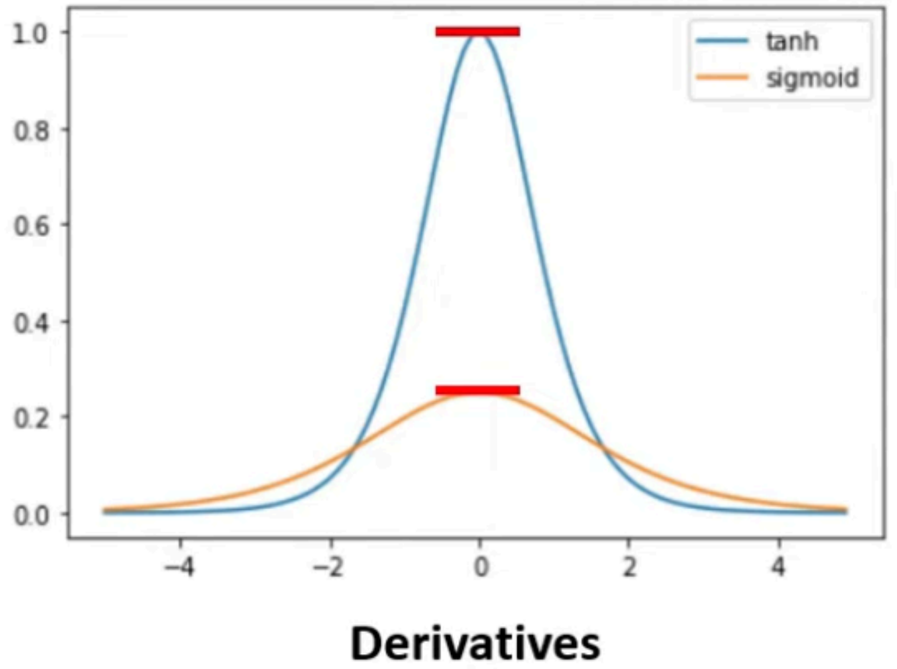
$$W = W - \text{alpha} * \frac{\partial \text{cost}}{\partial W}$$

We need to take derivative of activation functions

$$\frac{\partial \tanh}{\partial x}$$

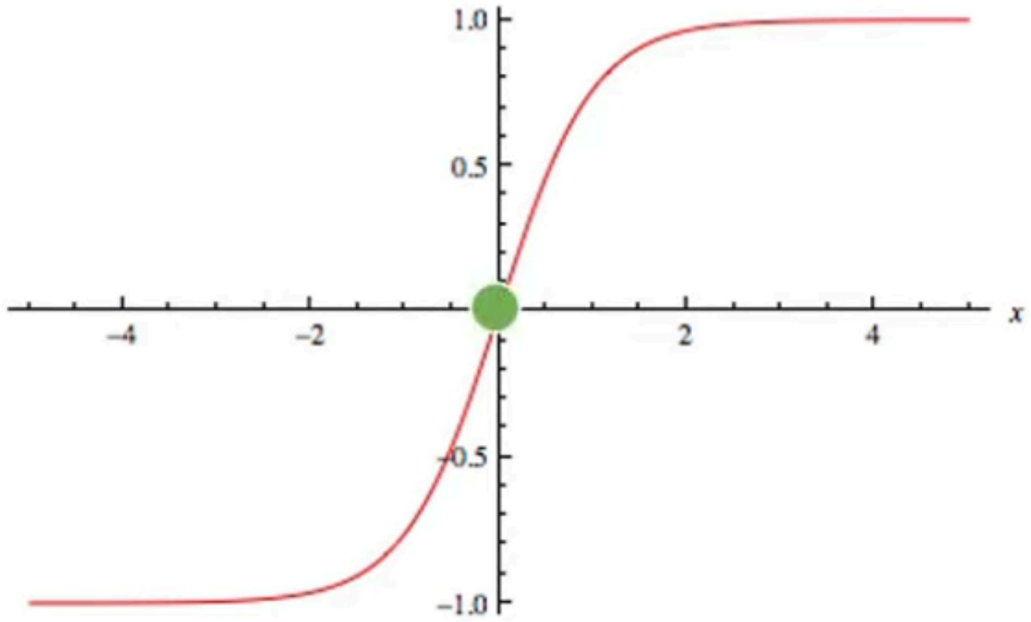
Types of Activation Functions

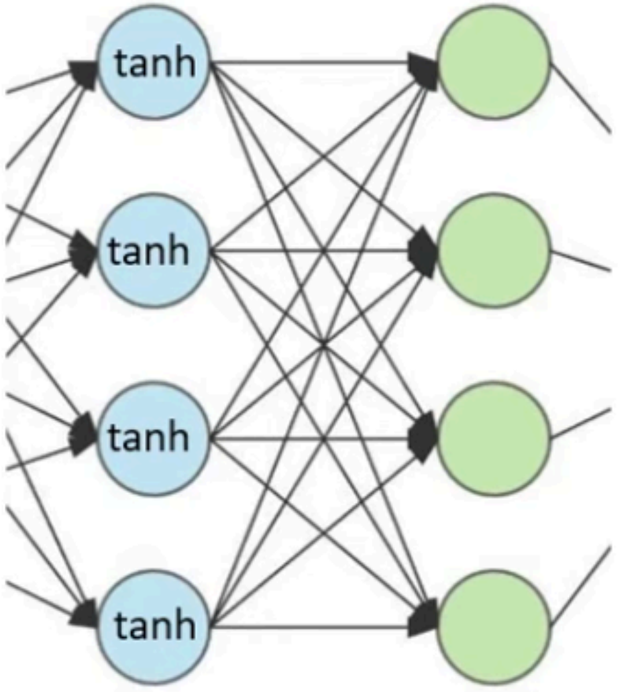
- **TanH Function vs Sigmoid Function**



Types of Activation Functions

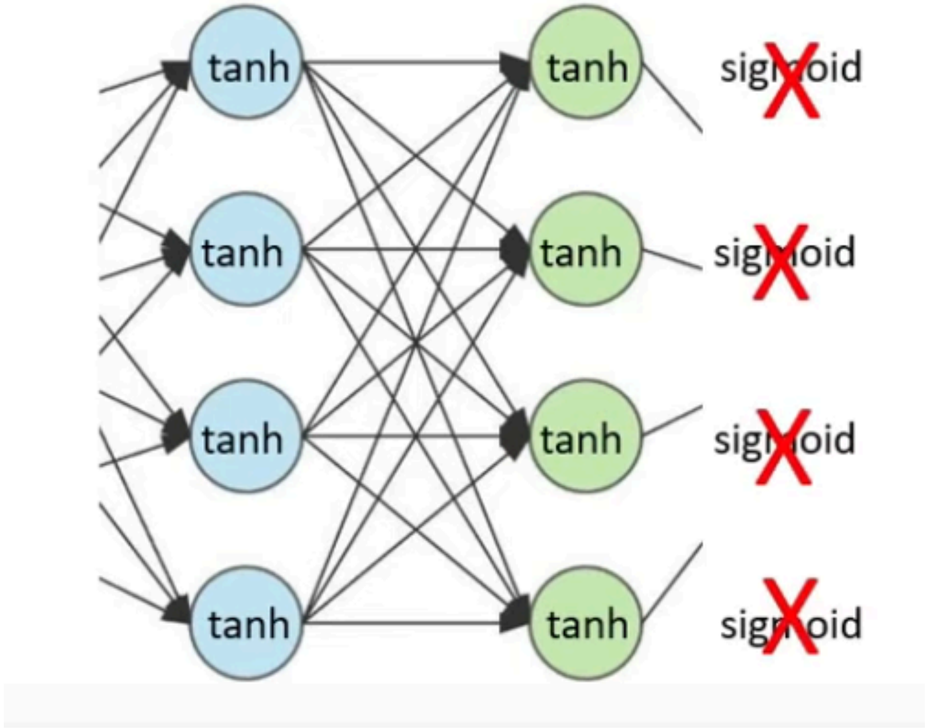
- **TanH Function:**
- Since average being 0, we can say “Data is more cenetered towards Zero”. Making learning easier.





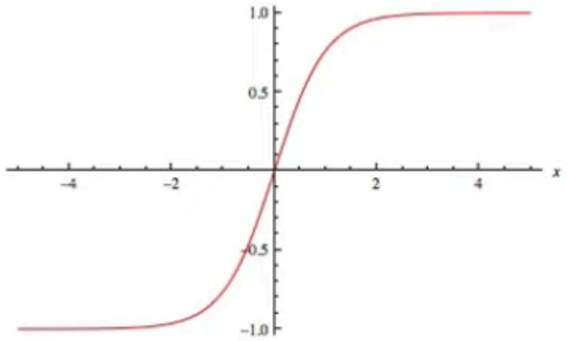
Types of Activation Functions

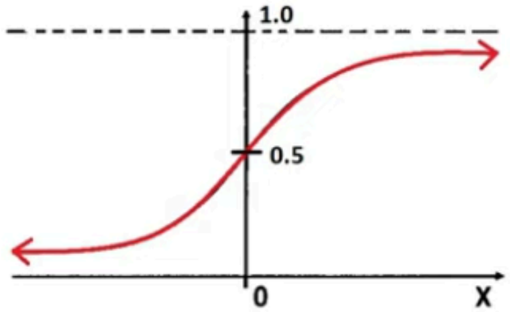
- **TanH Function vs Sigmoid Function**

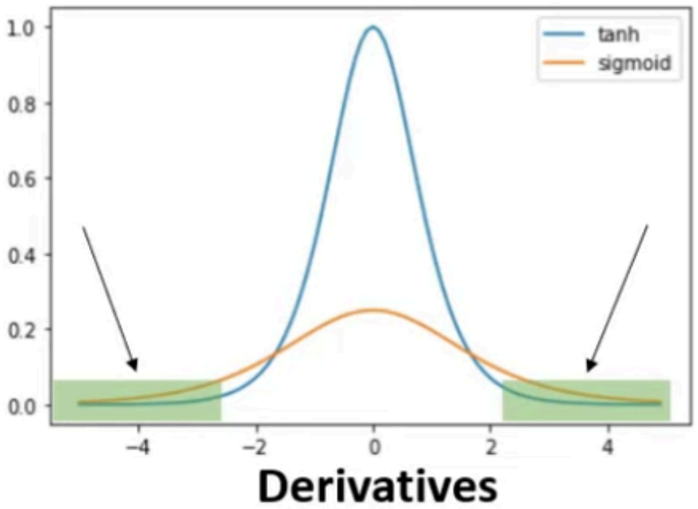


Types of Activation Functions

- **TanH Function vs Sigmoid Function**





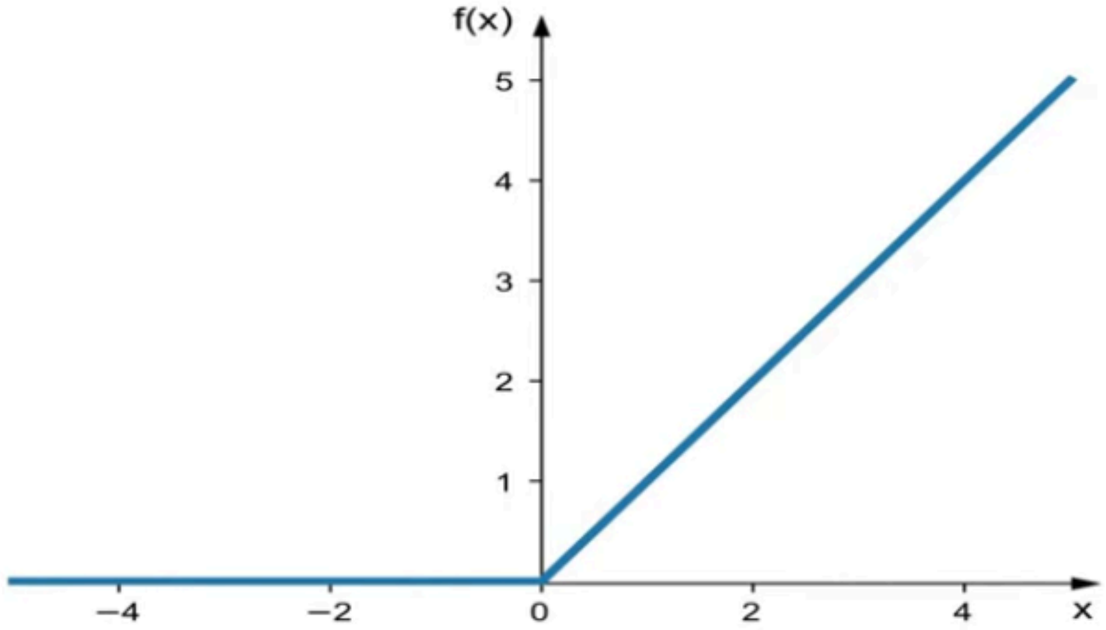


Smaller value of derivative leads to very slow learning

Vanishing Gradient Problem

Types of Activation Functions

- **ReLU (Rectified Linear Unit)**



Piece-wise Linear

Advantages of both linear and non-linear property

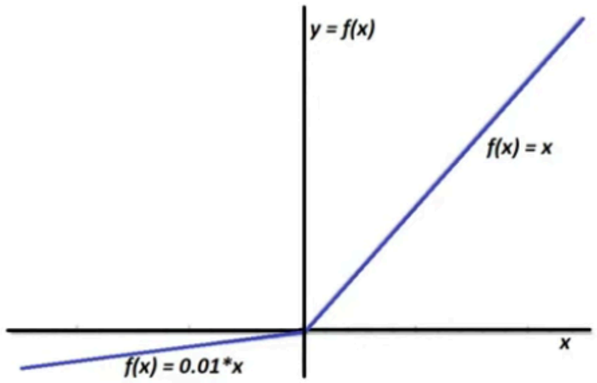
$$f(x) = max(0, x)$$

Overcome the Vanishing Gradient Problem

$$\frac{\partial f(x)}{\partial x} = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}$$

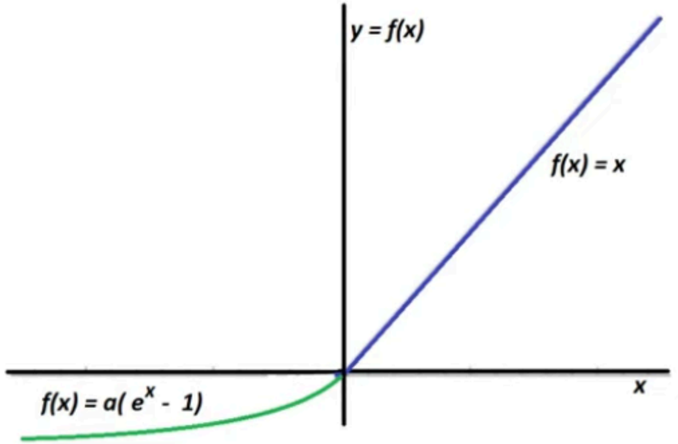
Types of Activation Functions

- **Variations of ReLU (Rectified Linear Unit)**



Leaky ReLU

$f(x) = max(0.01*x,\ x)$

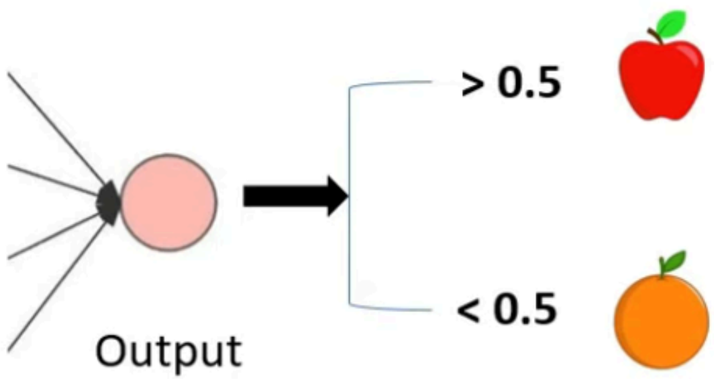


ELU (Exponential Linear Unit)

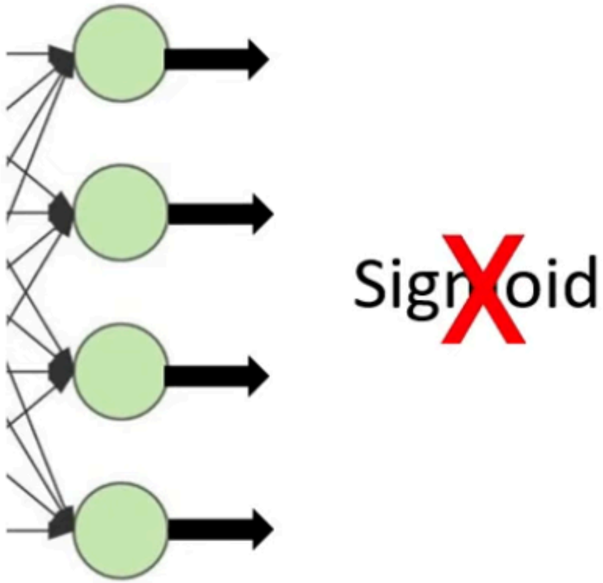
$f(x) = max(\ \alpha*(e^{x} - 1,\ x),\ x)$

Types of Activation Functions

- **Softmax Function**



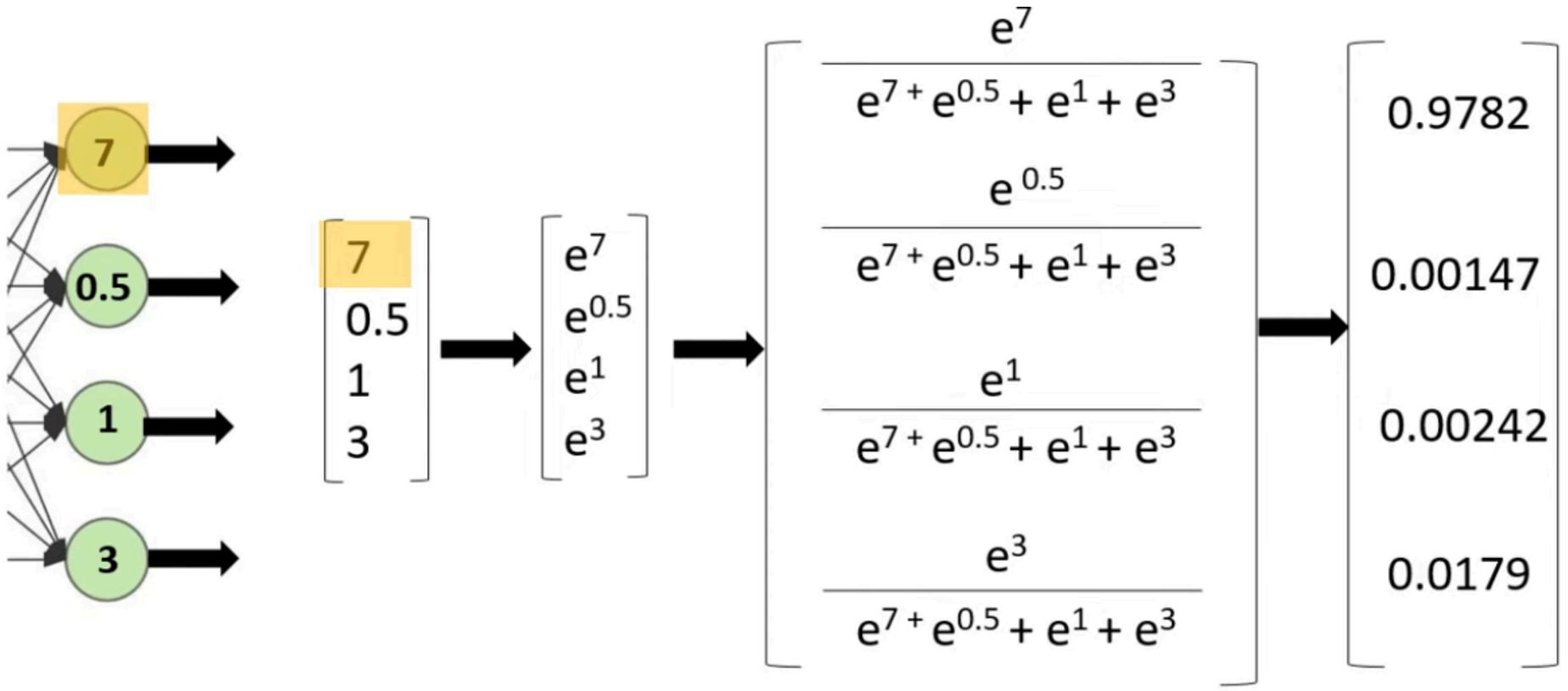
Binary Classification



Multi-Class Classification
??

Types of Activation Functions

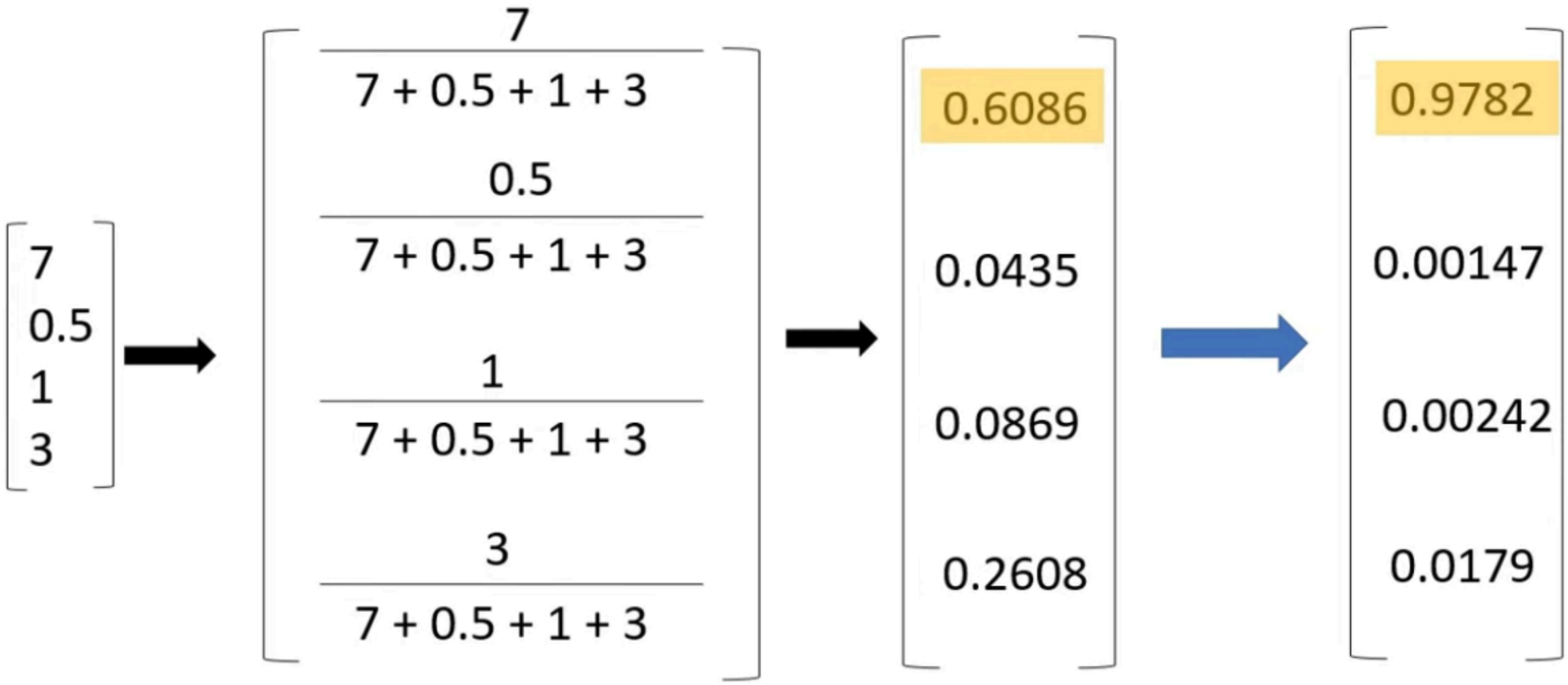
- **Softmax Function**



$$0.9782 + 0.00147 + 0.00242 + 0.0179 = 1$$

Types of Activation Functions

- **Softmax Function**



Exponent makes larger value more larger and smaller value more smaller

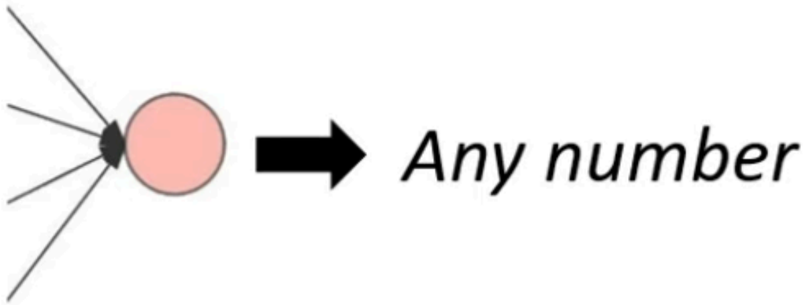
Types of Activation Functions

- **Softmax Function**

$$Softmax\ f_i(x) = \frac{e^{a_i}}{\sum_k e^{a_k}}$$

Activation Functions

What if our output is continious?

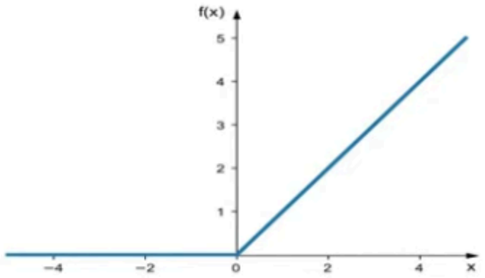


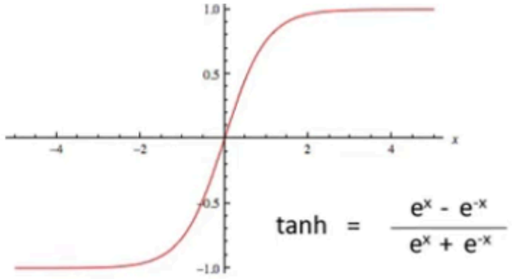
House Price Prediction

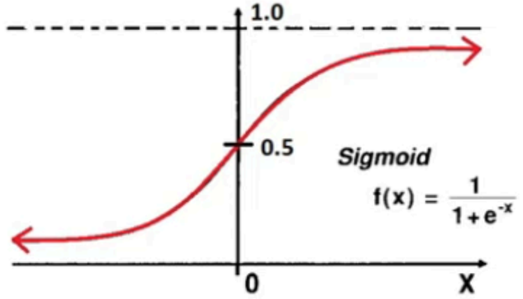
Price of the house can be anywhere from 0 to $1 million

No Activation Function in Output Neuron

Activation Functions - Conclusion

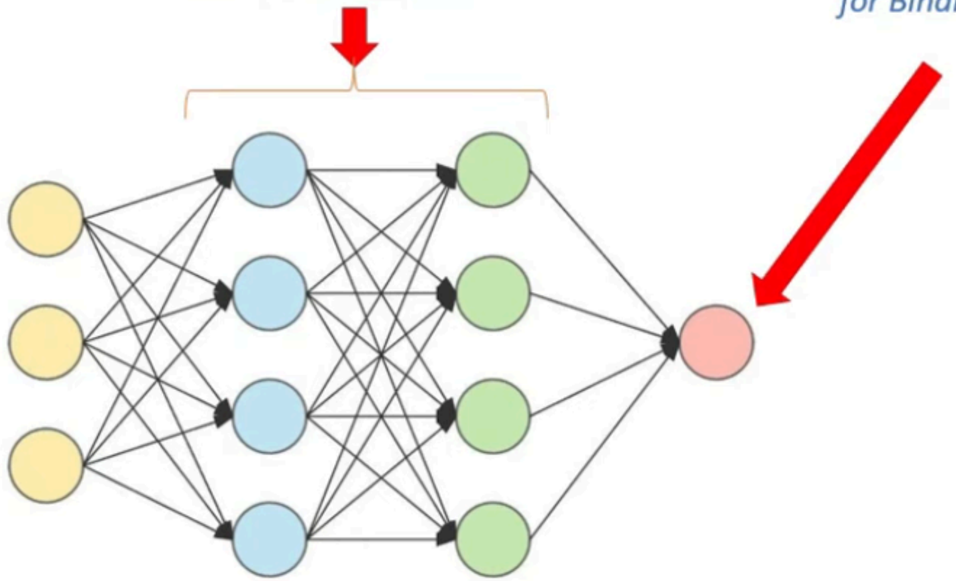




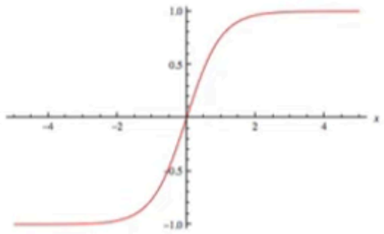


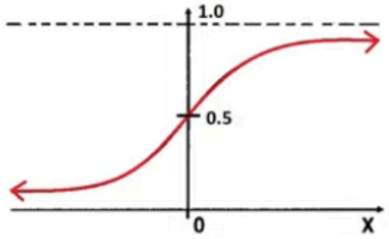
2) Either ReLU or tanh can be used in hidden layers

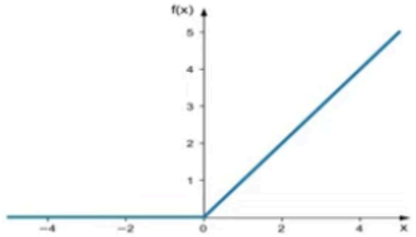
1) Sigmoid in output Neuron for Binary Classification

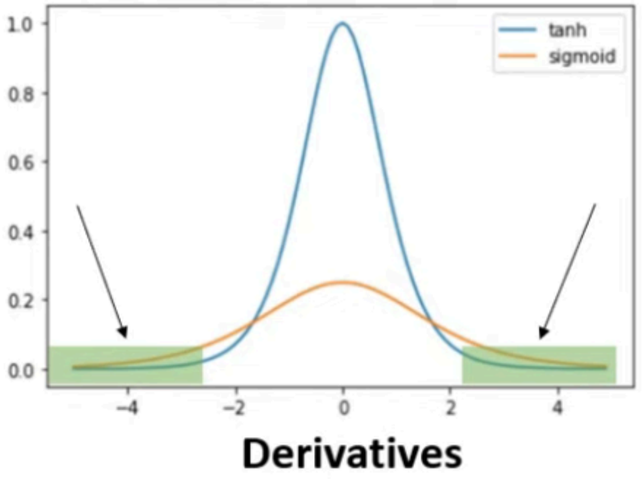


Activation Functions - Conclusion

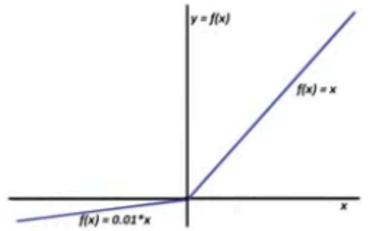


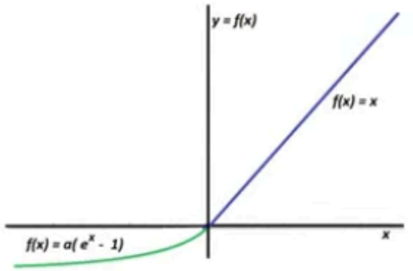






Vanishing Gradient Problem

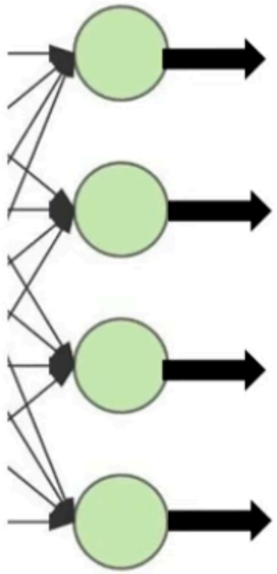




Activation Functions - Conclusion

- **Multi-Class Classification**

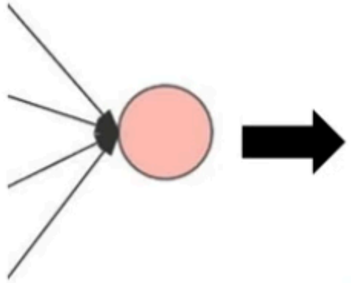
tanh or ReLU in hidden layers



$$\text{Softmax } f_i(x) = \frac{e^{a_i}}{\sum_k e^{a_k}}$$

Softmax in Output layer

- **Continious values Prediction**

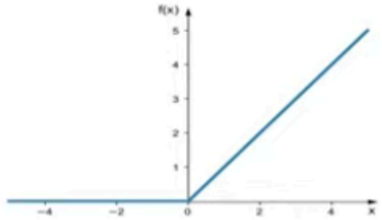


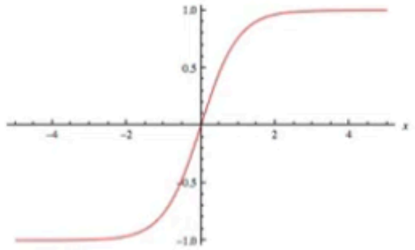
Any number

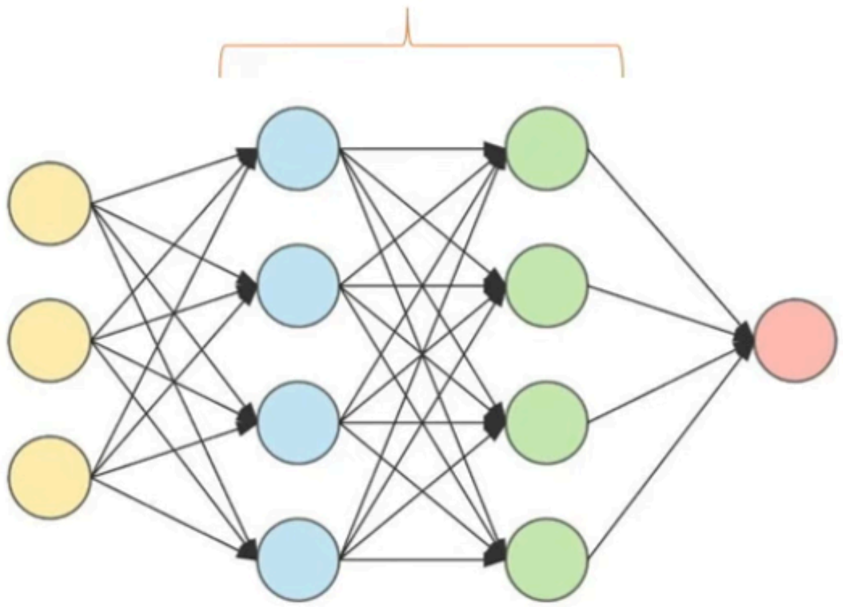
No Activation Function in Output Neuron

Activation Functions - Conclusion

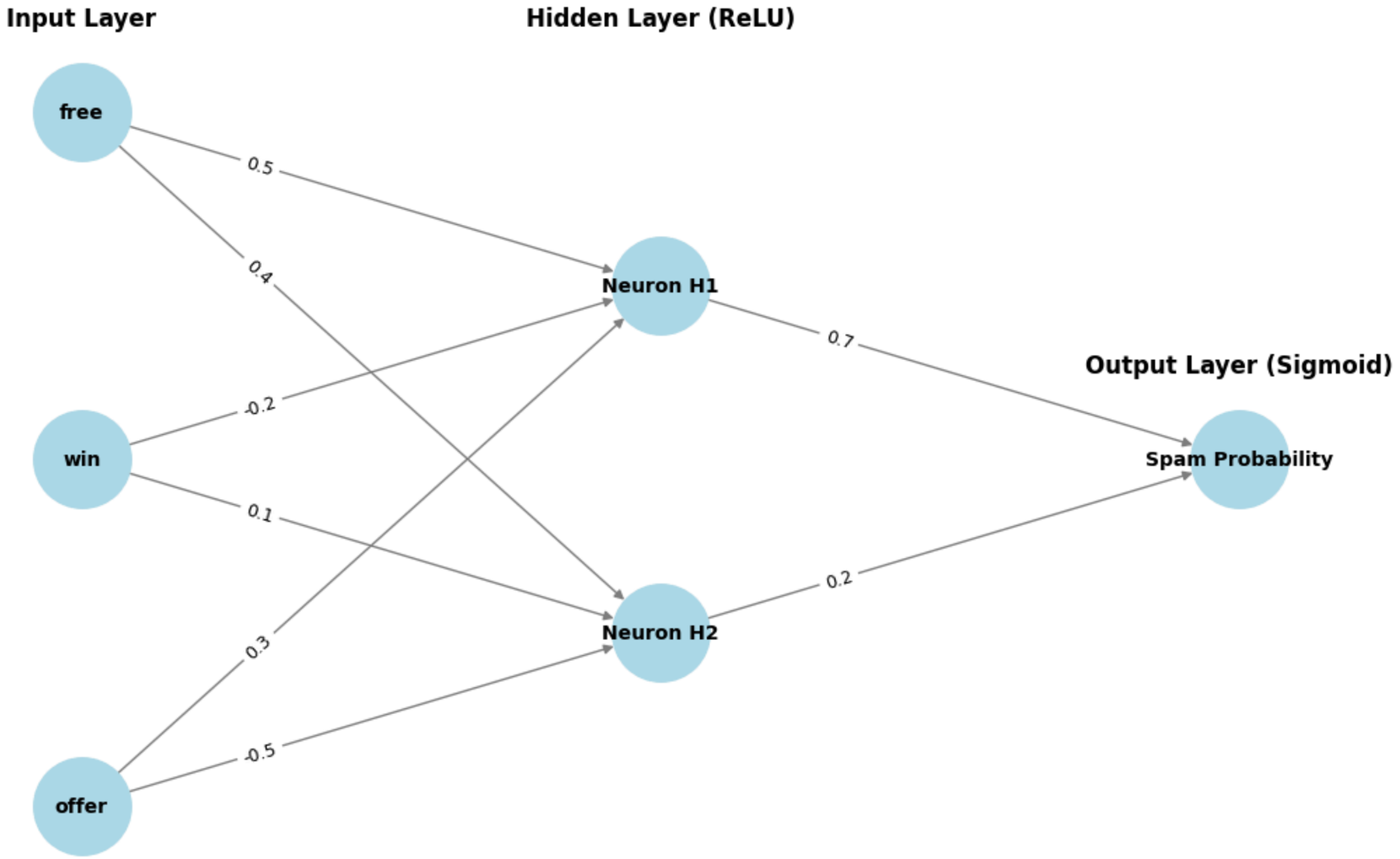
Try using both and see which gives better performance







Activation Functions - Conclusion



Cost Functions

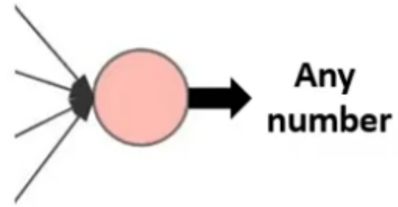
- **Cost Function:** is the error representation in machine learning.
- It shows how our model is predicting based on actual values.
- Lesser the cost function → Higher the accuracy of our model
- Higher the cost function → Lesser the accuracy of our model

Different Cost Functions of Different types of Problems

- Regression or Linear Regression
- Binary Classification
- Multi-class Classification

Cost Functions - Regression Problem

- error = | ai - yi |
- For M number of Observations
- Cost vs Loss
  -



House Price Prediction

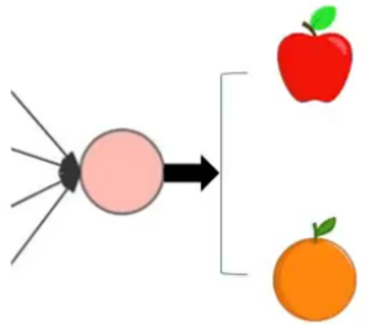
$$\frac{1}{m}\sum_{i=1}^{m}|a_i - y_i|$$

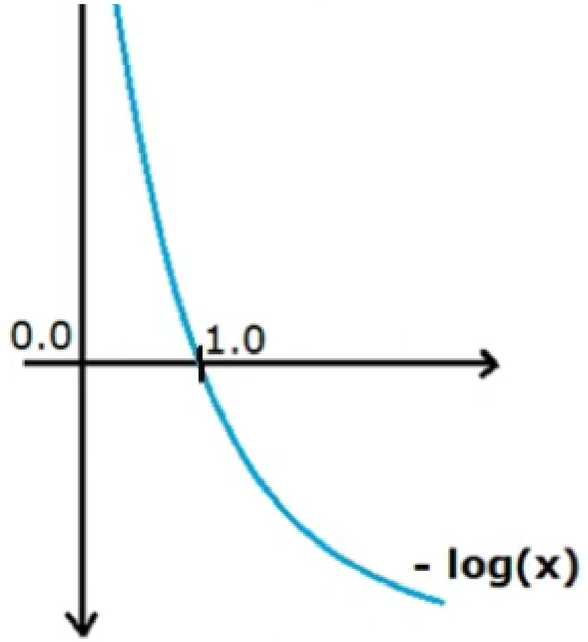
$$\frac{1}{2m}\sum_{i=1}^{m}|a_i - y_i|^2$$

Cost Functions - Binary Classification

- error = -[yi log(ai) + (1-yi)log(1-ai)]
- a.k.a Binary Cross Entropy

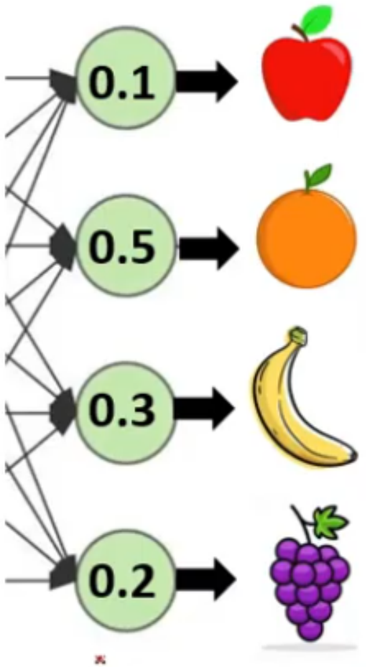
$$-\frac{1}{m}\sum_{i=1}^{m}[y_i \log(a_i) + (1-y_i)\log(1-a_i)]$$





Cost Functions - Multi-class Classification

- One hot encoding of y
- Matrices of y and a
- error = - [yi log(ai) + ..]
- a.k.a Categorical Cross Entropy

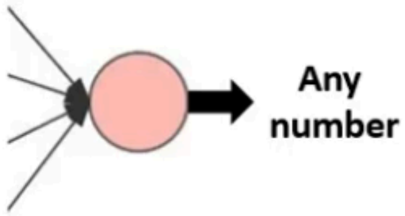


$$\text{cost} = -\sum_{j=0}^{M}\sum_{i=0}^{N}(y_{ij} * log(a_{ij}))$$

- N = total no. of neurons in the output layer
- M = total observations

Cost Functions - Conclusion

1.) Regression

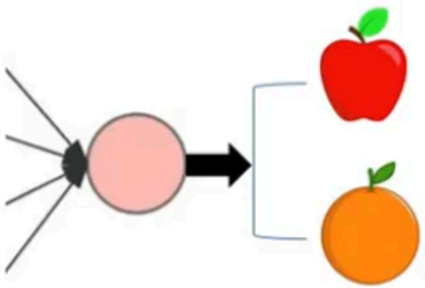


$$\text{Cost} = \frac{1}{m}\sum_{i=1}^{m}|a_i - y_i|$$

or

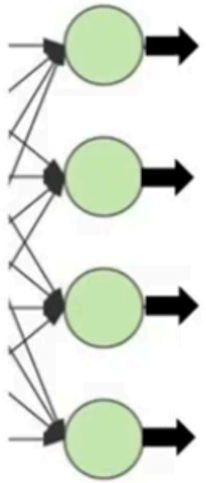
$$\text{Cost} = \frac{1}{2m}\sum_{i=1}^{m}[a_i - y_i]^2$$

2.) Binary Classification



$$\text{Cost} = -\frac{1}{m}\sum_{i=1}^{m}[y_i * log(a_i) + (1 - y_i) * log(1 - a_i)]$$

3.) Multi-class classification



$$\text{cost} = -\sum_{j=0}^{M}\sum_{i=0}^{N}(y_{ij} * log(a_{ij}))$$

Types of Neural Networks

- **Feedforward Networks:** A feedforward neural network is a simple artificial neural network architecture in which data moves from input to output in a single direction.
- **Single layer Perceptron:** A single-layer perceptron consists of only one layer of neurons . It takes inputs, applies weights, sums them up and uses an activation function to produce an output.
- **Multi layer Perceptron:** MLP is a type of feedforward neural network with three or more layers, including an input layer, one or more hidden layers and an output layer. It uses nonlinear activation functions.
- **Convolutional Neural Networks (CNN):** A Convolutional Neural Network (CNN) is a specialized artificial neural network designed for image processing. It employs convolutional layers to automatically learn hierarchical features from input images, enabling effective image recognition and classification.
- **Recurrent Neural Networks (RNN):** An artificial neural network type intended for sequential data processing is called a Recurrent Neural Network (RNN). It is appropriate for applications where contextual dependencies are critical such as time series prediction and natural language processing, since it makes use of feedback loops which enable information to survive within the network.
- **Long Short Term Memory (LSTM):** LSTM is a type of RNN that is designed to overcome the vanishing gradient problem in training RNNs. It uses memory cells and gates to selectively read, write and erase information.

Python Walkthrough

https://github.com/merledu/genify/tree/main/lecture8